LeadFlow

SWEN 383: SOFTWARE DESIGN PRINCIPLES AND PATTERNS

TEAM MEMBERS

Donat Bojaj • Aron Gorani • Abdelaziz Dabab • Vlerton Bytyqi

PROFESSOR

Valon Raca

PRESENTATION DATE: Wednesday, October 28, 2026

GITHUB REPOSITORY: github.com/00dnn/LeadFlow

1. Executive Summary & Problem Statement

In dedicated sales organizations, operational friction between outbound appointment setters and sales closers frequently results in lost context, dropped leads, and unaccountable pipelines. Setters qualify prospects and book consultations, but once handed off, traditional workflows lack a real-time feedback loop. Setters rarely receive immediate insight into whether their booked appointments are converted, no shows, or required subsequent follow-up. This disconnect degrades qualification quality, clouds commission visibility, and slows conversion velocity.

LeadFlow is an engineered, web-based appointment and lead management system designed to eliminate this communication gap. By enforcing structured role-based handoffs, transparent real-time state tracking, and an immutable consultation audit history, LeadFlow ensures complete operational cohesion across the customer acquisition lifecycle.

2. Stakeholder Roles & Workflow Governance

Role Identity	Core Responsibilities	System Capabilities & Access Controls
Appointment Setter TOP-OF-FUNNEL	Captures qualified lead demographic data, prospect service interests, preferred scheduling windows, and assigns prospective clients to designated closers.	Modal appointment entry, automatic closer assignment, KPI tracking (Total Submitted, Conversion Yield), and real-time read-only outcome debrief review.
Sales Closer BOTTOM-OF-FUNNEL	Executes customer consultations, logs interaction summaries, and drives leads through defined commercial qualification stages toward contract execution.	Dedicated closer workboard, appointment detail inspection, status transition authorization (Pending → Won/Lost), and immutable consultation history logging.

3. System Scope & Architecture Foundations

For the upcoming implementation phase, LeadFlow will be built as a client-side web application using standard **HTML5, CSS3, and vanilla JavaScript (ES6+)**, using browser **localStorage** for data persistence. We deliberately avoid third-party frameworks to focus purely on core object-oriented software design patterns.

Core Architectural Patterns:

Model-View-Controller (MVC): Separates user interface events, application workflow, and lead data logic.

Repository Pattern & DIP: Decouples business logic from storage by having `LeadService` depend on the `ILeadRepository` interface rather than directly on `localStorage`.

Observer Pattern: Uses an `EventEmitter` to broadcast state updates (e.g., when a lead is created or updated) across views without tightly coupling components.

.

4. System Architecture & Static Structural Model

The structural design of LeadFlow adheres strictly to separation of concerns. The domain model, persistence mechanisms, business service orchestration, and presentation layer are decoupled through

formal interfaces and event-driven communication channels, adhering to SOLID principles.

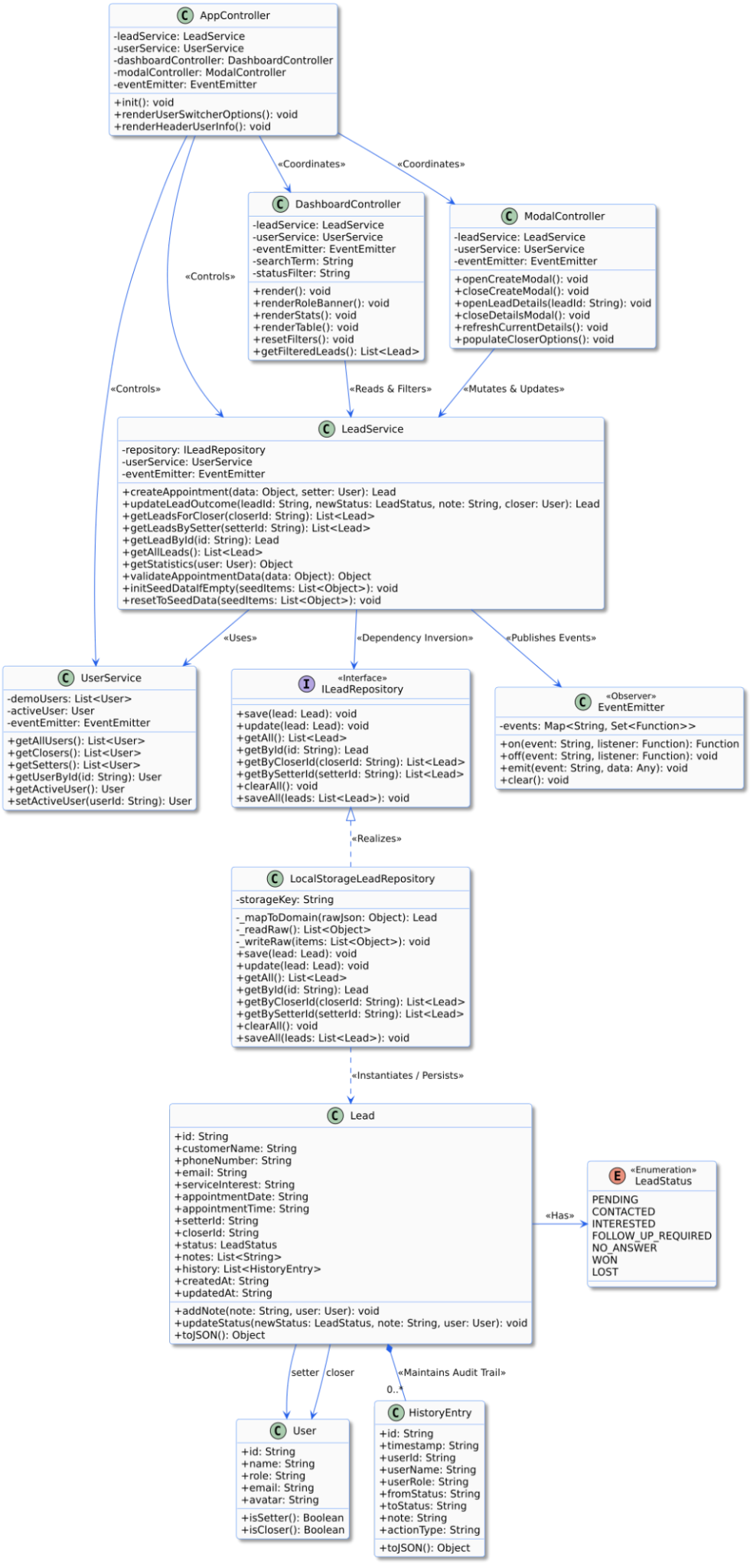

AppController
-leadService: LeadService
-userService: UserService
-dashboardController: DashboardController
-modalController: ModalController
-eventEmitter: EventEmitter
+init(): void
+renderUserSwitcherOptions(): void
+renderHeaderUserInfo(): void
«Coordinates»
«Coordinates»
«Controls»
«Controls»
DashboardController
-leadService: LeadService
-userService: UserService
-eventEmitter: EventEmitter
-searchTerm: String
-statusFilter: String
+render(): void
+renderRoleBanner(): void
+renderStats(): void
+renderTable(): void
+resetFilters(): void
+getFilteredLeads(): List<Lead>
ModalController
-leadService: LeadService
-userService: UserService
-eventEmitter: EventEmitter
+openCreateModal(): void
+closeCreateModal(): void
+openLeadDetails(leadId: String): void
+closeDetailsModal(): void
+refreshCurrentDetails(): void
+populateCloserOptions(): void
«Reads & Filters»
«Mutates & Updates»
LeadService
-repository: ILeadRepository
-userService: UserService
-eventEmitter: EventEmitter
+createAppointment(data: Object, setter: User): Lead
+updateLeadOutcome(leadId: String, newStatus: LeadStatus, note: String, closer: User): Lead
+getLeadsForCloser(closerId: String): List<Lead>
+getLeadsBySetter(setterId: String): List<Lead>
+getLeadById(id: String): Lead
+getAllLeads(): List<Lead>
+getStatistics(user: User): Object
+validateAppointmentData(data: Object): Object
+initSeedDataIfEmpty(seedItems: List<Object>): void
+resetToSeedData(seedItems: List<Object>): void
«Uses»
«Dependency Inversion»
«Publishes Events»
UserService
-demoUsers: List<User>
-activeUser: User
-eventEmitter: EventEmitter
+getAllUsers(): List<User>
+getClosers(): List<User>
+getSetters(): List<User>
+getUserById(id: String): User
+getActiveUser(): User
+setActiveUser(userId: String): User
«Interface»
ILeadRepository
+save(lead: Lead): void
+update(lead: Lead): void
+getAll(): List<Lead>
+getById(id: String): Lead
+getByCloserId(closerId: String): List<Lead>
+getBySetterId(setterId: String): List<Lead>
+clearAll(): void
+saveAll(leads: List<Lead>): void
«Observer»
EventEmitter
-events: Map<String, Set<Function>>
+on(event: String, listener: Function): Function
+off(event: String, listener: Function): void
+emit(event: String, data: Any): void
+clear(): void
«Realizes»
LocalStorageLeadRepository
-storageKey: String
-_mapToDomain(rawJson: Object): Lead
-_readRaw(): List<Object>
-_writeRaw(items: List<Object>): void
+save(lead: Lead): void
+update(lead: Lead): void
+getAll(): List<Lead>
+getById(id: String): Lead
+getByCloserId(closerId: String): List<Lead>
+getBySetterId(setterId: String): List<Lead>
+clearAll(): void
+saveAll(leads: List<Lead>): void
«Instantiates / Persists»
Lead
+id: String
+customerName: String
+phoneNumber: String
+email: String
+serviceInterest: String
+appointmentDate: String
+appointmentTime: String
+setterId: String
+closerId: String
+status: LeadStatus
+notes: List<String>
+history: List<HistoryEntry>
+createdAt: String
+updatedAt: String
+addNote(note: String, user: User): void
+updateStatus(newStatus: LeadStatus, note: String, user: User): void
+toJSON(): Object
«Has»
«Enumeration»
LeadStatus
PENDING
CONTACTED
INTERESTED
FOLLOW_UP_REQUIRED
NO_ANSWER
WON
LOST
setter
closer
«Maintains Audit Trail»
0..*
User
+id: String
+name: String
+role: String
+email: String
+avatar: String
+isSetter(): Boolean
+isCloser(): Boolean
HistoryEntry
+id: String
+timestamp: String
+userId: String
+userName: String
+userRole: String
+fromStatus: String
+toStatus: String
+note: String
+actionType: String
+toJSON(): Object

Figure 4.1: LeadFlow Structural Class Diagram (MVC, Repository Realization, and Event Observer Bus)

4.1 Component Decomposition & Design Realization

Domain Aggregates (Lead, HistoryEntry, LeadStatus): The Lead entity acts as an aggregate root encapsulating customer attributes, assignment pointers (setterId, closerId), and an ordered collection of immutable HistoryEntry objects detailing state transitions.

Repository Decoupling (ILeadRepository & LocalStorageLeadRepository): Enforcing the Dependency Inversion Principle (DIP), LeadService interacts exclusively with the abstract ILeadRepository interface, isolating domain logic from physical storage drivers.

Decoupled Observer Bus (EventEmitter): Implements a publish-subscribe pattern to broadcast domain lifecycle events (lead:created, lead:updated), synchronizing views without cross-controller dependencies. Presentation Controllers (AppController, DashboardController, ModalController): Manage interface lifecycles, user switches, role-based card filtering, and modal submissions, delegating all domain logic to the service layer.

5. Dynamic Behavioral Model

The dynamic behavior of LeadFlow coordinates asynchronous DOM interactions, synchronous domain integrity validation, persistence serialization, and reactive view synchronization across distinct business scenarios

5.1 Sequence 1: Lead Ingestion & Closer Assignment

Scenario 1 traces the end-to-end execution flow when an Appointment Setter books a qualified prospect. The workflow enforces data validation, aggregate entity creation, persistent storage, and event-driven view refresh.

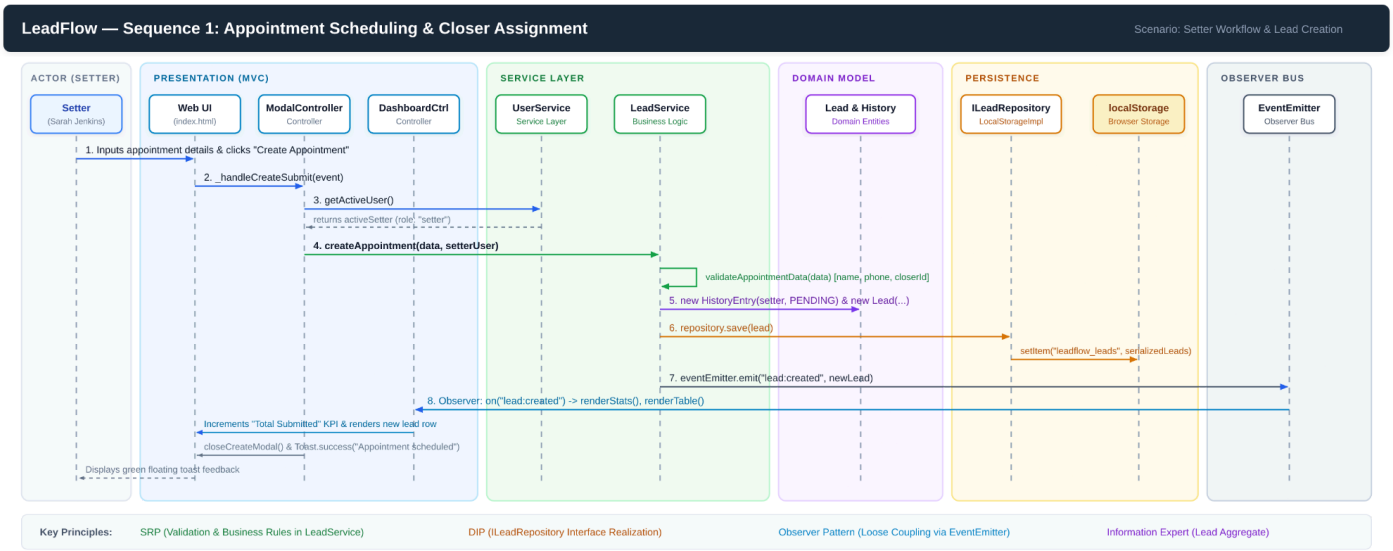


Figure 6.1: Sequence Diagram 1 — Appointment Scheduling, Closer Routing, and Observer Broadcast

5.1.1 Interaction Walkthrough & Architectural Rules

1. UI Trigger & Active Context: The Setter submits appointment criteria via Web UI. ModalController intercepts the DOM event and resolves the current authenticated actor via UserService.getActiveUser(). 2. Single Responsibility Validation (SRP): LeadService.createAppointment() executes validateAppointmentData(), verifying telephone formatting, appointment scheduling, and valid closer ID assignment before entity instantiation.

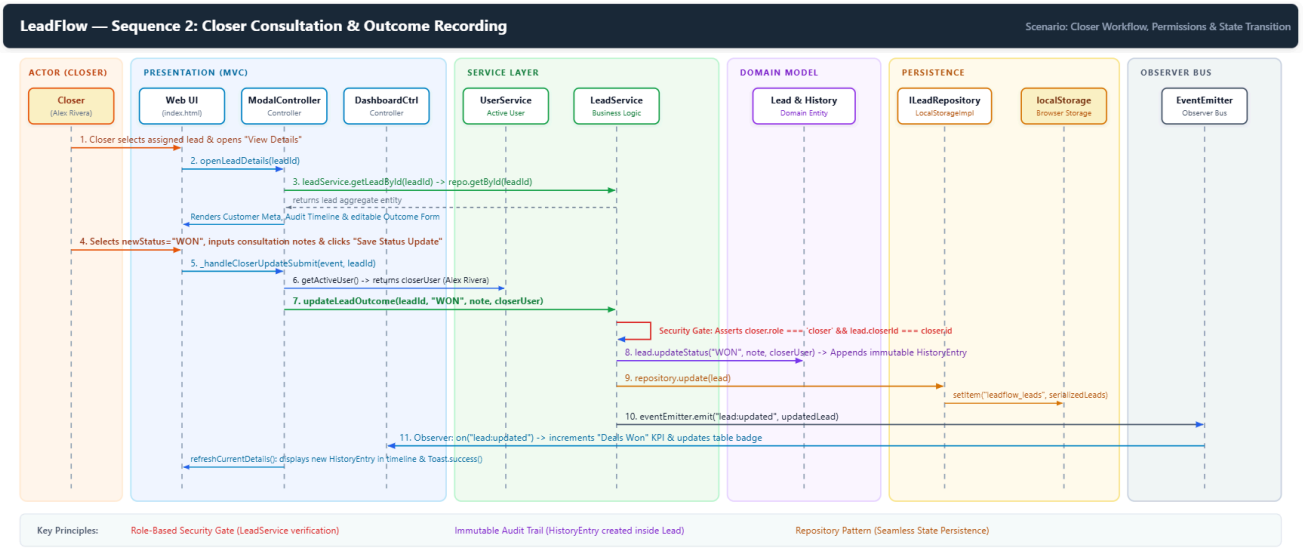
3. Aggregate Construction: LeadService instantiates the Lead domain model with default status PENDING, immediately appending an initial HistoryEntry establishing the creation audit baseline.

4. **Persistent State Handoff:** The entity is dispatched to ILeadRepository.save(lead), which serializes the domain aggregate into browser storage via LocalStorageLeadRepository.

5. **Asynchronous Decoupled Notification:** LeadService emits "lead:created" through the EventEmitter. The listening DashboardController re-renders KPI metric counters and appends the new table row seamlessly.

5.2 Sequence 2: Closer Consultation & Outcome Recording

Sequence 2 models the sales consultation outcome workflow. It highlights role-based authorization verification, domain aggregate status transitions, immutable history recording, and localized dashboard re rendering.



5.2.1 Interaction Walkthrough & Security Rules

1. **Dossier Hydration:** The Closer selects an assigned appointment. ModalController requests the aggregate entity via LeadService.getLeadById(), displaying customer demographics, historical notes, and an editable consultation form.

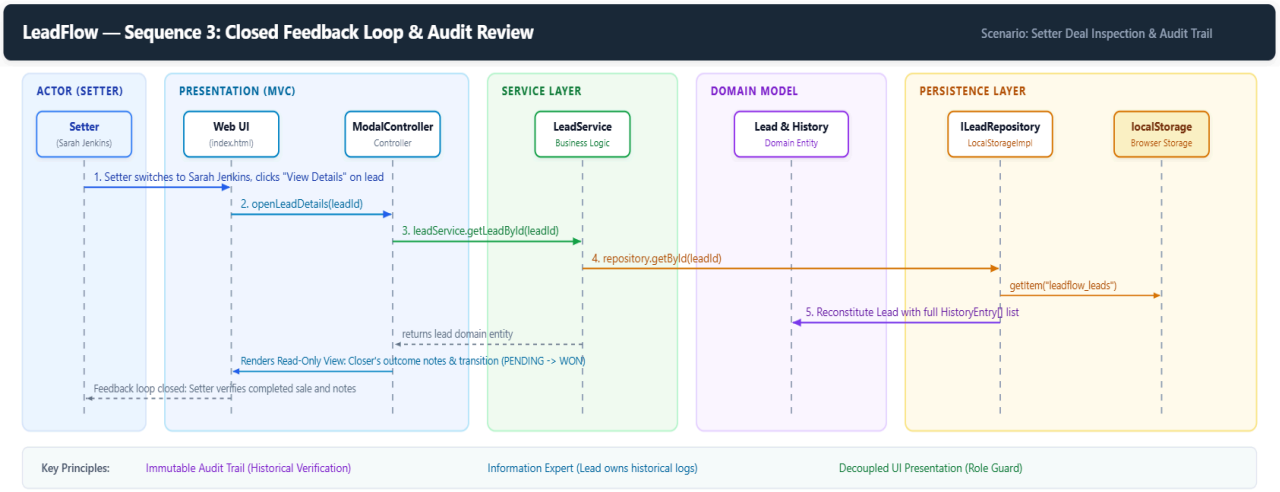
2. **Role-Based Security Gate:** Upon submission of a new status (e.g., WON) with outcome notes, LeadService.updateLeadOutcome() enforces a critical security gate: assert(closer.role === 'closer' && lead.closerId === closer.id), rejecting unauthorized mutations.

3. **Information Expert Audit Append:** The Lead domain aggregate invokes updateStatus(), recording a new immutable HistoryEntry capturing timestamp, transition delta (PENDING → WON), Closer ID, and notes.

4. **Synchronous Storage & Event Broadcast:** The updated entity is stored via repository.update(lead), followed by an EventEmitter.emit("lead:updated", updatedLead) signal, prompting immediate UI feedback and metric updates.

5.3 Sequence 3: Closed Feedback Loop & Audit Review

Sequence 3 captures LeadFlow's core architectural differentiator: the closed feedback loop. This scenario illustrates how appointment setters inspect historical outcomes, review closer consultation notes, and refine qualification criteria.



5.3.1 Feedback Transparency & Read-Only Protection

1. Workflow Context Reversal: The Appointment Setter (e.g., Sarah Jenkins) logs into LeadFlow to inspect submitted leads. The dashboard presents distinct status badges (WON , LOST , CONTACTED).

2. Entity Reconstitution: When the setter clicks "View Details", LeadService.getLeadById() retrieves raw serialized JSON via LocalStorageLeadRepository, reconstituting the full Lead aggregate with complete HistoryEntry[] entries.

3. Role-Guarded UI Projection: ModalController recognizes the active user's role as setter. The outcome editing form is disabled, presenting a read-only timeline of the closer's notes, timestamps, and customer responses. 4. Closed Feedback Value Realization: The setter gains direct insight into conversion blockers (e.g., "Customer budget disqualified", "Needs follow-up on pricing"), closing the operational loop and improving future lead quality.

6. Anticipated Design Risks & Proactive Design Patterns

Risk 1: "Bloated UI" & High Coupling from Mixing Event Handlers with Business Logic

Challenge: In web applications, a frequent pitfall is embedding input validation, lead creation, and status calculations directly inside HTML DOM click handlers. As warned in the course (Week 4, Slide 14), this creates a "Bloated UI" with low cohesion and high coupling, making core business logic untestable and fragile to UI layout changes.

Proactive Pattern & Principle — GRASP Controller & Single Responsibility Principle (SRP): We separate concerns by routing UI input through dedicated coordinating controllers (ModalController, DashboardController). The controllers capture DOM events and immediately delegate to LeadService, ensuring presentation code remains thin and business rules stay cohesive in the domain layer.

Risk 2: Hard-Wired Storage Coupling & Violation of Dependency Inversion

Challenge: If LeadService directly calls browser localStorage, high-level business rules become tightly coupled to low level browser storage APIs (Week 3, DIP violation). This makes automated unit testing difficult because tests cannot substitute a mock storage driver, and any future database migration would require rewriting business logic.

Proactive Pattern & Principle — Dependency Inversion (DIP) & GRASP Pure Fabrication: We define an abstract ILeadRepository interface and inject it into LeadService. We fabricate LocalStorageLeadRepository as a Pure Fabrication—an artificial class whose sole responsibility is handling persistence. This fulfills Protected Variations, insulating domain logic from storage details.

Risk 3: "God Objects" & Centralized Service Class Bloat

Challenge: As system capabilities expand, core application services risk accumulating excessive responsibilities—such as user session management, notification delivery, domain rules, and data formatting. Accumulating unrelated responsibilities into a single monolithic service creates an unmaintainable "God Object" with low cohesion and fragile dependency chains.

Proactive Pattern & Principle — Single Responsibility Principle (SRP) & GRASP High Cohesion: We maintain high cohesion by restricting LeadService strictly to lead aggregate management and status transition verification. Secondary responsibilities are decomposed into specialized, standalone modules (such as UserService for identity management and EventEmitter for asynchronous event handling), ensuring clear separation of concerns and manageable service boundaries.

7. Scope Boundaries

Scope Classification	Functional & Architectural Boundaries
In-Scope Capabilities	Appointment Intake & Validation, Role-Based Views & Security Gates, Lead Lifecycle State Engine, Audit History Logging, Decoupled Architecture, Local Persistence, and Filtering & Metrics.
Out-of-Scope Capabilities	Remote Backend Server & Database, Production Authentication & Security, External Calendar Integrations, Automated Telephony & Messaging, and Payment Processing & Invoicing.

8. Project Sign-off

Course & Term

SWEN 383 • Fall 2026

Phase 1 Architecture Deliverable

Team Designation

Group: Marimangat